In other words, `remove`'s running time is dominated by `findMin`; this also means that in a 2-D space the amortized running time for `remove` would be `O()`.

9.3.5 Nearest Neighbor

We are now ready to study the most interesting operation provided by k-d trees, the nearest neighbor (*NN*) search. First, we are going to restrict to the case where we search for the single closest point in the dataset with respect to a target point (which, in general, doesn't have to be contained in the same dataset); later we will generalize this operation to return an arbitrary number `m`[10] of points such that no other point in the dataset is closer to the target.

In a brute-force scenario, we would have to compare each point in the dataset to our target, compute their relative distances, and keep track of the smallest one: exactly the way we search for an element in an unsorted array.

